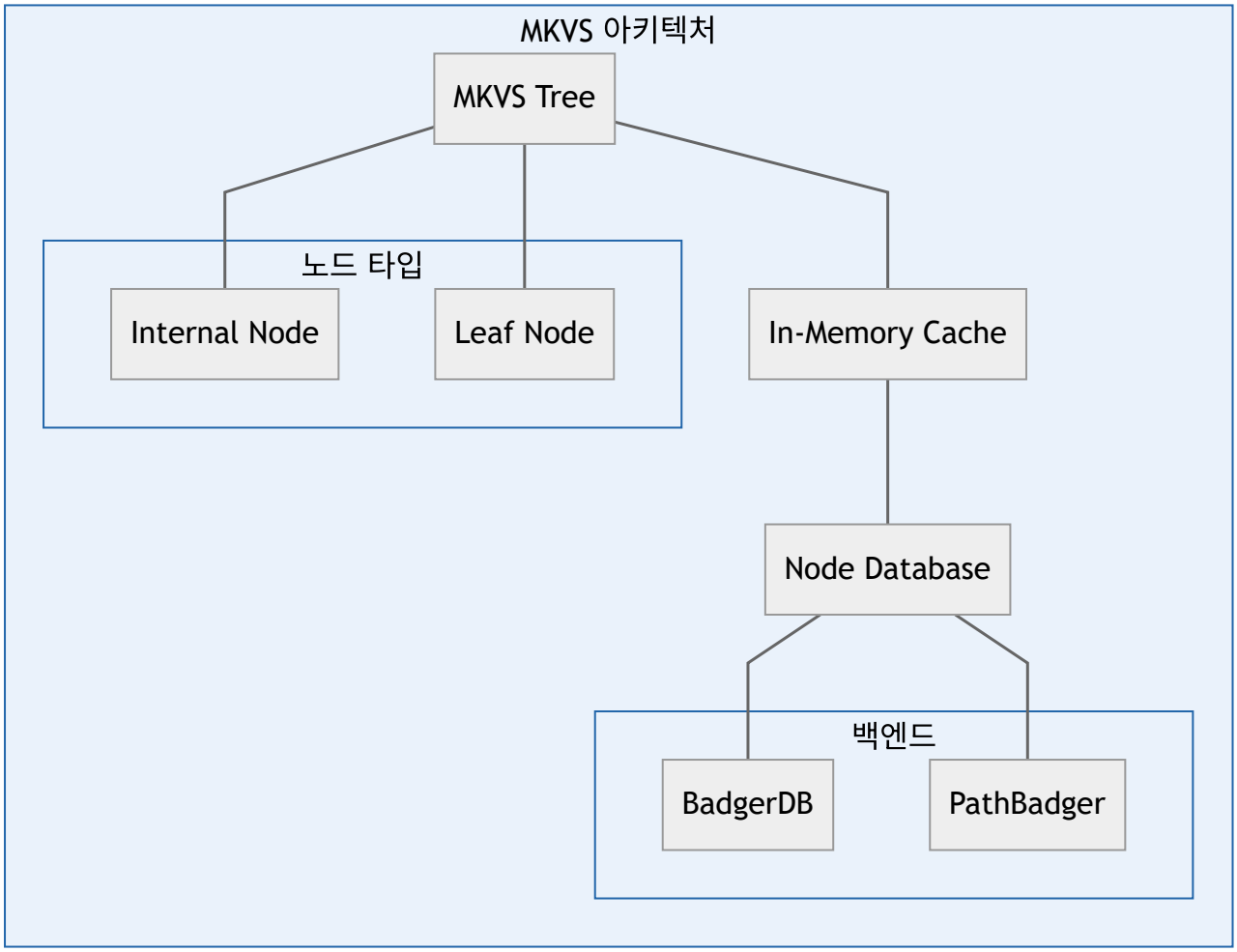
MKVS 스토리지 (Merklized Key-Value Store)

소개

MKVS(Merklized Key-Value Store)는 Oasis Core의 핵심 스토리지 시스템으로, Merkle Tree 기반의 검증 가능한 키-값 저장소입니다. MKVS는 블록체인의 상태 데이터를 안전하고 효율적으로 저장하며, 암호학적 증명을 통해 데이터 무결성을 보장합니다.

아키텍처 개요

핵심 구성 요소



주요 특징

- **Merkle Tree 구조**: 모든 노드가 암호학적 해시로 연결된 트리 구조
- **검증 가능성**: 부분 데이터만으로도 전체 상태의 무결성 검증 가능
- **효율적 동기화**: 차이점만 전송하여 빠른 상태 동기화 지원
- **압축 저장**: 중복 제거와 압축을 통한 효율적 저장 공간 활용

Merkle Tree 구조

노드 타입

1. Internal Node (내부 노드)

```
type InternalNode struct {
Hash hash.Hash // 노드의 해시값
Label Key // 엣지 레이블
LabelBitLength Depth // 레이블 비트 길이
Clean  bool  // 정리 상태 플래그
LeafNode *Pointer // 리프 노드 포인터
Left *Pointer // 왼쪽 자식 포인터
Right *Pointer // 오른쪽 자식 포인터
}
```

특징:

- 두 개의 자식 노드를 가짐 (Left, Right)
- 선택적으로 리프 노드를 포함할 수 있음

- 레이블을 통해 키 공간을 분할

2. Leaf Node (리프 노드)

```
type LeafNode struct {
Clean  bool  // 정리 상태 플래그
Hash hash.Hash // 노드의 해시값
Key Key // 키
Value []byte  // 값
}
```

특징:

- 실제 키-값 쌍을 저장
- 트리의 말단 노드
- 해시 계산에 키와 값이 모두 포함됨

해시 계산 방식

Internal Node 해시

```
func (n *InternalNode) UpdateHash() {
leafNodeHash := n.LeafNode.GetHash()
leftHash := n.Left.GetHash()
rightHash := n.Right.GetHash()
labelBitLength := n.LabelBitLength.MarshalBinary()
```

```
n.Hash.FromBytes(
[]byte{PrefixInternalNode},
labelBitLength,
n.Label[:],
leafNodeHash[:],
leftHash[:],
rightHash[:],
)
}
```

Leaf Node 해시

```
func (n *LeafNode) UpdateHash() {
var  keyLen, valueLen [4]byte
binary.LittleEndian.PutUint32(keyLen[:], uint32(len(n.Key)))
binary.LittleEndian.PutUint32(valueLen[:], uint32(len(n.Value)))

n.Hash.FromBytes(
[]byte{PrefixLeafNode},
keyLen[:],
n.Key[:],
valueLen[:],
n.Value[:]
)
```

```
}
```

스토리지 노드 구조

포인터 시스템

```
type Pointer struct {
Clean  bool  // 정리 상태
Hash hash.Hash // 해시 참조
Node Node // 실제 노드 (메모리에 로드된 경우)
LRU *list.Element // LRU 캐시 요소
DBInternal DBPointer // 데이터베이스 내부 메타데이터
}
```

포인터의 역할:

- **지연 로딩**: 필요할 때만 노드를 메모리에 로드
- **캐시 관리**: LRU 방식으로 메모리 사용량 제어
- **해시 참조**: 디스크에 저장된 노드를 해시로 참조

압축 메커니즘

1. 노드 압축

- **Path Compression**: 단일 자식을 가진 내부 노드들을 압축
- **Label Compression**: 공통 접두사를 레이블로 압축

2. 직렬화 압축

```go
// 컴팩트 직렬화 (증명용)
func (n *InternalNode) CompactMarshalBinaryV1() ([]byte, error) {
data := make([]byte, 0, 1+DepthSize+len(n.Label)+1)
data = append(data, PrefixInternalNode)
data = append(data, n.LabelBitLength.MarshalBinary()...)
data = append(data, n.Label...)
data = append(data, PrefixNilNode) // 해시 포인터 제외
return  data, nil
}
```

MKVS API

기본 인터페이스

1. 읽기 전용 트리

```go
type ImmutableKeyValueTree interface {
// 키로 값 조회
Get(ctx context.Context, key []byte) ([]byte, error)
// 이터레이터 생성
NewIterator(ctx context.Context, options ...IteratorOption) Iterator
}
```

2. 읽기-쓰기 트리

```go
type KeyValueTree interface {
ImmutableKeyValueTree
// 키-값 쌍 삽입
Insert(ctx context.Context, key, value []byte) error
// 기존 키 제거 (이전 값 반환)
RemoveExisting(ctx context.Context, key []byte) ([]byte, error)
// 키 제거
Remove(ctx context.Context, key []byte) error
}
```

3. 전체 트리 인터페이스

```go
type Tree interface {
KeyValueTree
ClosableTree
syncer.ReadSyncer
// 접두사 프리페치
PrefetchPrefixes(ctx context.Context, prefixes [][]byte, limit  uint16) error
// 쓰기 로그 적용
ApplyWriteLog(ctx context.Context, wl writelog.Iterator) error
// 알려진 루트로 커밋
CommitKnown(ctx context.Context, root node.Root) (writelog.WriteLog, error)
```

```
// 커밋 (새 루트 생성)
Commit(ctx context.Context, namespace common.Namespace, version  uint64, options
}
```

사용 예제

1. 기본 MKVS 트리 생성

```
// 새 MKVS 트리 생성
tree := mkvs.New(nil, ndb, node.RootTypeState)

// 키-값 쌍 삽입
err := tree.Insert(ctx, []byte("key1"), []byte("value1"))
if  err != nil {
return  err
}

// 값 조회
value, err := tree.Get(ctx, []byte("key1"))
if  err != nil {
return  err
}
```

2. 기존 루트에서 트리 생성

```go
// 기존 루트로 트리 생성
root := node.Root{
Namespace: namespace,
Version: version,
Type: node.RootTypeState,
Hash: rootHash,
}

tree := mkvs.NewWithRoot(nil, ndb, root)

// 데이터 조회
value, err := tree.Get(ctx, []byte("existing_key"))
```

3. 트리 이터레이션

```go
// 모든 키-값 쌍 순회
iter := tree.NewIterator(ctx)
defer iter.Close()

for iter.Rewind(); iter.Valid(); iter.Next() {
key := iter.Key()
value := iter.Value()
fmt.Printf("Key: %s, Value: %s\n", key, value)
```

```
}
```

4. 접두사 기반 조회

```
// 특정 접두사로 시작하는 키들 프리페치
prefixes := [][]byte{
[]byte("user:"),
[]byte("config:"),
}

err := tree.PrefetchPrefixes(ctx, prefixes, 1000)
if err != nil {
return err
}

// 접두사 이터레이터 사용
iter := tree.NewIterator(ctx, mkvs.IteratorPrefetch([]byte("user:")))
defer iter.Close()

for iter.Rewind(); iter.Valid(); iter.Next() {
// user: 접두사를 가진 모든 키 처리
processUserData(iter.Key(), iter.Value())
```

```
}
```

데이터 검증 방식

Merkle Proof 생성

```
// 특정 키에 대한 증명 생성
proof, err := syncer.NewProof(ctx, tree, []byte("target_key"))
if  err != nil {
return  err
}

// 증명 검증
isValid := proof.Verify(rootHash)
```

증명 구조

```
type Proof struct {
// 증명에 포함된 노드들
Entries []ProofEntry `json:"entries"`
}

type ProofEntry struct {
Key []byte  `json:"key,omitempty"`
```

```
Value []byte `json:"value,omitempty"`
Hash []byte `json:"hash,omitempty"`
}
```

성능 최적화

1. 캐시 설정

```
// 캐시 용량 설정
tree := mkvs.New(
nil,
ndb,
node.RootTypeState,
mkvs.Capacity(5000, 16*1024*1024), // 5000개 노드, 16MB 값
)
```

2. 배치 작업

```
// 여러 작업을 배치로 처리
for _, operation := range operations {
switch operation.Type {
case "insert":
tree.Insert(ctx, operation.Key, operation.Value)
```

```
case  "remove":
tree.Remove(ctx, operation.Key)
}
}

// 한 번에 커밋
writeLog, newRoot, err := tree.Commit(ctx, namespace, version)
```

3. 프리페치 최적화

```
// 자주 접근하는 키들을 미리 로드
commonPrefixes := [][]byte{
[]byte("account:"),
[]byte("balance:"),
[]byte("nonce:"),
}

tree.PrefetchPrefixes(ctx, commonPrefixes, 10000)
```

오류 처리

일반적인 오류 타입

```
// 노드를 찾을 수 없음
```

```go
if errors.Is(err, mkvs.ErrNodeNotFound) {
// 노드가 존재하지 않음
}

// 트리가 닫힘
if errors.Is(err, mkvs.ErrClosed) {
// 트리가 이미 닫혔음
}

// 알려진 루트 불일치
if errors.Is(err, mkvs.ErrKnownRootMismatch) {
// 예상 루트와 계산된 루트가 다름
}
```

복구 전략

```go
func handleMKVSError(err error, tree mkvs.Tree) error {
switch {
case errors.Is(err, mkvs.ErrClosed):
// 새 트리 인스턴스 생성
return createNewTree()
case errors.Is(err, mkvs.ErrKnownRootMismatch):
// 상태 동기화 재시도
```

```
return resyncState()
default:
return err
}
}
```

모니터링 및 디버깅

트리 상태 덤프

```
// 로컬 메모리의 트리 구조 덤프
var buf bytes.Buffer
tree.DumpLocal(ctx, &buf, node.Depth(10)) // 최대 깊이 10

fmt.Println("Tree structure:", buf.String())
```

메트릭 수집

```
// 트리 크기 정보
func getTreeMetrics(tree mkvs.Tree) map[string]interface{} {
return map[string]interface{}{
"root_type": tree.RootType().String(),
"cache_size": getCacheSize(tree),
```

```
"pending_operations": getPendingOperations(tree),

}

}
```

MKVS는 Oasis Core의 상태 관리를 위한 핵심 구성 요소로, 효율적이고 안전한 데이터 저장과 검증을 제공합니다. Merkle Tree 기반의 구조를 통해 부분 검증이 가능하며, 다양한 최적화 기법을 통해 높은 성능을 달성합니다.